

Contest Platform with Live Leaderboard

Database Implementation & Core Functionalities (MongoDB + Redis)

Team	Upayan Mazumder (24BDS0367), Aditya Rawat (24BCE2992), Shloak Sinha (24BDS0378)
Stack	MongoDB (replica set) + Redis, FastAPI backend

1. Database Implementation

Collection	Key fields
users	name, email, passwordHash, role, totalScore
teams	name, memberIds[], totalScore
contests	title, startTime, endTime, status, problemIds[], participants[]
problems	contestId, type (mcq/coding/subjective), difficulty, points
submissions	contestId, problemId, submittedBy, status, score

Relationships:

- Contest → Problems
- Contest → Participants (users/teams)
- User/Team → Submissions
- Team → Users
- Submission → Problem

Sample data loaded:

Collection	Records
users	15
teams	5
contests	5
problems	15
submissions	48
Total	88

Rubric needs 50+ records. We have 88, generated by a seed script.

2. CRUD Operations

Operation	Implemented for
Create	Users, teams, contests, problems, submissions
Read	All 5 collections (single record + filtered list)
Update	Users, teams, contests, problems, submissions
Delete	Users, teams, contests, problems, submissions

Example (create + read a user):

```
POST /users {"name":"Doc User","email":"doc@rankstack.io","password":"secret123"}
-> {"id":"6ab3f9a5...", "name":"Doc User", ...}

GET /users/6ab3f9a5...
-> {"id":"6ab3f9a5...", "name":"Doc User", ...}
```

3. Advanced NoSQL Features

3.1 Aggregation

Aggregation	Purpose
Contest leaderboard	total score + rank per contestant
Problem statistics	average score per difficulty level
Submission statistics	count of submissions by status, per contest
Problems solved per user	distinct correct problems across contests
Team performance	combine team members' submissions into one score

Sample result: {name: "Team Lavender Trial", totalScore: 195.0}

3.2 Indexing

- Unique index on `users.email`
- Index on `problems.contestId`
- Compound index on `submissions (contestId, submittedBy)`

Checked with `explain()` : queries use the index (IXSCAN), not a full collection scan.

3.3 Transactions

Used when one action needs to update more than one document at the same time:

- New submission + attempt count on the problem
- Submission score update + user/team total score
- Team membership change + back-reference on the user

3.4 Redis

- Live leaderboard (updated as scores change)
- Login sessions (expire automatically)
- Rate limiting on submissions

Tested: after 10 submissions in a minute, the 11th is blocked.

4. Review 2 Checklist

Criterion	Marks	Status
Database implementation	2	Done
CRUD operations	3	Done
Advanced NoSQL features	5	Done

5. Repository

Git repository at the project workspace (`rankstack/`). Includes the app code, the seed script, and a demo script that runs through the CRUD, aggregation, and transaction examples above.